YadYar Lite - Phase 2 Report

T-05: Retrieval-Augmented Generation and the Study of Hallucination

Taha Amini and Eiliya Yavari

AI and Expert Systems - Spring 1404-1405 | Instructor: Dr. Koohzadi

1. Project Question

How well can a lightweight retrieval-augmented generation (RAG) system find answer evidence in a small document collection, generate answers supported by that evidence, and avoid answering when sufficient evidence is unavailable?

The pipeline is evaluated in two parts. First, the retrieval evaluation measures whether one of the retrieved chunks contains the reference answer. Second, the generation evaluation measures the quality of the final answers and whether the generator returns `UNANSWERABLE` for questions that cannot be answered from the available evidence.

Separating these two stages helps distinguish retrieval failures from generation failures.

2. Dataset

The project uses a reproducible subset of the SQuAD 2.0 development set. SQuAD 2.0 contains Wikipedia passages with both answerable and unanswerable questions, making it suitable for studying question answering, abstention, and hallucination.

The prepared subset contains:

Item	Value
Source contexts	56
Total context words	7,144
Retrieval chunks	146
Chunk size	70 words
Chunk overlap	15 words
Answerable questions	50
Unanswerable questions	50
Random seed	42

Each question includes its SQuAD ID, question text, source context ID, reference answers, and an `is_answerable` label. The preprocessing script and generated dataset files are included in the repository.

3. Baseline System

The baseline is a lightweight local RAG pipeline with three main stages:

```
Question
   -> BGE query embedding
   -> ChromaDB Top-3 retrieval
   -> Qwen2.5 generation
   -> answer or UNANSWERABLE
```

3.1 Retrieval

The retriever uses the pretrained `BAAI/bge-base-en-v1.5` embedding model. Each chunk and question is converted into a dense vector, and the vectors are stored in ChromaDB using cosine distance.

For every question, the three most similar chunks are retrieved:

```
Top-k = 3
```

No embedding model training or fine-tuning is performed.

3.2 Generation

The generator uses the pretrained `qwen2.5:3b` model through a local Ollama service.

The model receives the question and the text of the three retrieved chunks. Its prompt instructs it to:

- use only the retrieved context;
- avoid outside knowledge;
- return the shortest supported answer;
- return exactly `UNANSWERABLE` when the context is insufficient.

The generation settings are:

Setting	Value
Model	`qwen2.5:3b`
Temperature	0
Seed	42
Maximum generated tokens	64

The full evaluation can be reproduced from the repository root with:

```
python -m rag_engine.evaluation.evaluate_retrieval
python -m rag_engine.evaluation.evaluate_generation
```

Detailed results are stored in:

```
rag_engine/evaluation/retrieval_results.json
rag_engine/evaluation/generation_results.json
```

4. Evaluation Metrics

Two main metrics are used, with hallucination rate reported as an additional abstention diagnostic.

4.1 Evidence Recall@3

Evidence Recall@3 evaluates retrieval on the 50 answerable questions. A retrieval is successful when at least one normalized reference answer appears in one of the three retrieved chunks.

$$\text{Evidence Recall@3} = \frac{\text{answerable questions with evidence in Top-3}}{\text{all answerable questions}}$$

This metric is stricter than checking only the source context ID because a chunk from the correct context may still exclude the sentence containing the answer.

4.2 Answer F1

Answer F1 evaluates the generated answers for the 50 answerable questions. It calculates token-level overlap between the prediction and each reference answer and uses the highest score.

Before comparison, the evaluator:

- converts text to lowercase;
- removes punctuation;
- removes the English articles `a`, `an`, and `the`;
- removes extra whitespace.

F1 gives partial credit when a generated answer contains only part of the reference answer or includes additional words.

4.3 Hallucination Rate

For an unanswerable question, the expected output is `UNANSWERABLE`. Any normal answer generated for such a question is counted as a hallucination.

$$\text{Hallucination Rate} = \frac{\text{unanswerable questions given a normal answer}}{\text{completed unanswerable questions}}$$

This is an operational definition for this experiment. It measures failure to abstain, rather than verifying every factual statement in the generated text.

5. Baseline Results

The retrieval evaluator processed all 50 answerable questions, and the generation evaluator processed all 100 questions. No generation request failed.

Metric	Result	Interpretation
Evidence Recall@3	**0.98**	Evidence was retrieved for 49 of 50 answerable questions.
Answer F1	**0.6825**	Generated answers had moderate average token overlap with the references.
Hallucination Rate	**0.44**	The model answered 22 of 50 unanswerable questions instead of abstaining.
Generation errors	**0**	All 100 generation requests completed successfully.

The strong Evidence Recall@3 shows that retrieval was usually successful. However, the lower Answer F1 and the 44% hallucination rate show that correct retrieval alone does not guarantee a correct or properly abstained final answer.

The main weakness of the complete pipeline is therefore generation and abstention behavior rather than retrieval coverage.

6. Simple Result Breakdowns

6.1 Retrieval by Question Length

The median answerable-question length was nine words. Questions were divided into short and long groups using this value.

Question group	Count	Evidence found	Evidence Recall@3
Short (`<= 9` words)	29	28	0.966
Long (`> 9` words)	21	21	1.000

The only retrieval failure occurred in the short-question group. The question used the phrase “this settlement” without naming the settlement, so its meaning depended strongly on the original paragraph. When used as an independent retrieval query, it did not contain enough identifying information.

This result does not prove that longer questions are always easier. The sample is small, but it shows how referentially ambiguous questions can reduce retrieval quality.

6.2 Answerable and Unanswerable Questions

Question type	Count	Outcome
Answerable	50	46 answered and 4 false abstentions
Unanswerable	50	28 correct abstentions and 22 hallucinations

The correct-abstention rate was:

$$\frac{28}{50} = 0.56$$

Among the 50 answerable questions:

- 26 received an Answer F1 of `1.0`;
- 12 received partial token overlap;
- 12 received an Answer F1 of `0.0`.

Some zero-F1 results represent genuine wrong answers, while others are caused by limitations of token-level matching.

7. Representative Error Analysis

The following examples were selected manually because they represent different failure patterns found in the evaluation results.

Question and system output	Error category	Explanation
“What present-day area was this settlement near?” Output: `UNANSWERABLE`	Retrieval failure	None of the Top-3 chunks contained the relevant passage mentioning Parris Island. The phrase “this settlement” was too ambiguous when separated from its original context.
“Who was Bill Aiken’s adopted mother?” Output: `UNANSWERABLE`	False abstention	The first retrieved chunk stated that Bill Aken was adopted by Lupe Mayorga. Despite retrieving the evidence, the generator abstained, possibly because of the `Aiken`/`Aken` spelling difference.
“Roughly, how much oxygen makes up the Earth crust?” Output: `abundant element by mass`	Answer extraction error	A retrieved chunk contained “making up almost half of the crust’s mass,” but the model selected a nearby descriptive phrase instead of the requested amount.
“Originally built with four layers, how many layers did DECnet evolve to have?” Output: `seven`	Ignored false premise	The evidence stated that DECnet was initially built with three layers. The model answered the final part of the question but ignored the contradiction in its premise instead of returning `UNANSWERABLE`.
“What happens when the immune system loses tolerance for tumor antigens?” Output: `It no longer attacks the tumor cells.`	Negation error	The evidence said that the immune system stops attacking when tolerance develops. The question reversed this condition by asking about losing tolerance, but the model copied the consequence without handling the negation.

These examples show that the dominant errors were not simple absence of retrieved information. In most cases, relevant text was available, but the generator failed to interpret the question-evidence relationship correctly.

False premises and negation were especially difficult. The generator often extracted a nearby phrase that looked like an answer instead of checking whether the evidence actually supported the complete question.

7.1 Metric Limitation Example

For the question “How many atoms combine to form dioxygen?”, the system predicted `2`, while the references included `two` and `two atoms`.

The answer is semantically correct, but the current normalization does not convert digits into number words. Therefore, its Answer F1 was `0.0`.

This example demonstrates that Answer F1 is useful but imperfect. It measures token overlap, not full semantic equivalence, so the reported value of `0.6825` slightly underestimates answer quality in cases involving equivalent surface forms.

8. Limitations

The experiment has several limitations:

- The corpus contains only 56 contexts and 100 questions, so the results cannot represent all RAG or open-domain question-answering tasks.
- Only one embedding model, one generator, one prompt, and one value of `Top-k` were evaluated.
- Fixed word-based chunking may split sentences or separate evidence from useful surrounding information.
- Evidence Recall@3 depends on normalized reference-answer occurrence and may miss valid paraphrased evidence.
- Token-level F1 does not recognize all semantically equivalent answers, such as `2` and `two`.
- Hallucination rate treats every normal answer to a labeled unanswerable question as a hallucination. It does not perform independent factual verification.
- Some SQuAD questions depend on their original paragraph and become ambiguous when used as standalone retrieval queries.
- A fixed seed and temperature improve reproducibility, but results may still change with a different Ollama or model version.

- The system has no separate evidence verifier for detecting false premises, contradictions, or unsupported generated claims.

Therefore, the reported metrics should be interpreted as results for this particular lightweight configuration, not as general performance claims about RAG systems.

9. Future Work

Two improvements are directly motivated by the observed errors.

9.1 Stronger Abstention and Evidence Verification

A verification step could compare the generated answer and the complete question against the retrieved evidence before returning the final response. This step could detect contradictions, false premises, and negation mismatches.

For example, the DECnet question claimed that the system originally had four layers, while the evidence stated three. A verifier could detect this conflict and force the system to return `UNANSWERABLE`.

A future experiment could compare the current prompt with a verification-oriented prompt or a small natural-language-inference model.

9.2 Query Rewriting or Reranking

Ambiguous questions containing expressions such as “this settlement” could be rewritten using their source context before retrieval. Alternatively, a reranker could examine the initial candidates more carefully.

This change would mainly target the single observed retrieval failure. Because retrieval already achieved an Evidence Recall@3 of 0.98, improving generation and abstention should remain the higher priority.

These improvements are proposed as Future Work and were not implemented in the current baseline.

10. Lightweight Demo

The project includes an end-to-end terminal demo in `main.py`.

After the dataset is ingested, the user enters a question. The program:

1. embeds the question;
2. retrieves the three most similar chunks;
3. displays the retrieved chunk information;
4. sends the chunks and question to `qwen2.5:3b`;
5. displays an answer, `UNANSWERABLE`, or `GENERATION_ERROR`.

The demo can be run with:

```
ollama pull qwen2.5:3b
python main.py
```

The demo is intentionally lightweight and runs locally. It does not require a web interface, external API key, model training, or deployment platform.

Useful demonstration examples include:

```
When did Ribault first establish a settlement in South Carolina?
```

Expected supported answer:

```
1562
```

An unrelated question with no evidence should demonstrate the abstention behavior:

```
Who is the current president of France?
```

Expected output:

```
UNANSWERABLE
```

The demo should also include at least one failure example from the error-analysis table to show both the strengths and limitations of the system.

11. Future Integration Note

The current component could later be connected to a learning-assistant interface. The upstream component would provide a student question, and the RAG component would return a generated answer together with its retrieved evidence.

A possible future input format is:

```
{
  "question": "When did Ribault establish the settlement?",
  "top_k": 3
}
```

A possible output format is:

```
{
  "answer": "1562",
  "status": "answered",
  "retrieved_chunks": [
    {
      "chunk_id": "43",
      "context_id": 17,
      "text": "Retrieved evidence text",
      "similarity_percentage": 69.53
    }
  ]
}
```

The `status` field could have one of these values:

```
answered
unanswerable
generation_error
```

Returning the evidence with the answer would allow a future dialogue system or user interface to display supporting text and distinguish a supported answer from abstention or an execution failure.

This format is only a proposed integration contract. The current demo displays the same information in the terminal rather than exposing an API.

12. AI Tool Usage

OpenAI ChatGPT was used to help organize parts of the preprocessing and evaluation code, review the experiment structure, and improve the report organization.

The team selected the dataset and models, executed the code locally, inspected the generated results, verified the reported metrics, selected representative errors, and is responsible for explaining all implementation and design decisions.

The dataset passages and reference answers come from SQuAD 2.0, not from ChatGPT.

13. Conclusion

The lightweight RAG baseline achieved an Evidence Recall@3 of `0.98`, showing that the retriever found reference-answer evidence for 49 of 50 answerable questions.

The final Answer F1 was `0.6825`. Although the system often generated answers with good reference overlap, it also made extraction errors and sometimes abstained despite having evidence.

The largest weakness was abstention on unanswerable questions. The model generated unsupported answers for 22 of 50 unanswerable questions, producing a hallucination rate of `0.44`.

The experiment therefore shows that strong retrieval does not by itself prevent hallucination. In this configuration, improving evidence interpretation and abstention behavior is more important than increasing retrieval coverage.

References

1. Rajpurkar, P., Jia, R., and Liang, P. (2018). Know What You Don’t Know: Unanswerable Questions for SQuAD. ACL.
2. SQuAD Explorer: https://rajpurkar.github.io/SQuAD-explorer/
3. BGE model page: https://huggingface.co/BAAI/bge-base-en-v1.5
4. Qwen2.5 model page: https://huggingface.co/Qwen/Qwen2.5-3B
5. ChromaDB documentation: https://docs.trychroma.com/
6. Ollama documentation: https://docs.ollama.com/